

Laporan Praktikum P5 - Inheritance, Generalization, Superclass, dan Subclass

Nama : Akhmad Satrio Cahyo Pratama

NRP : 3125522013

Studi : Teknik Informatika

1. Sprint Goal

Memperbaiki desain proyek P4 dengan mengidentifikasi class yang memiliki karakteristik serupa dan melakukan generalization menggunakan inheritance (superclass & subclass), tanpa membuat project baru.

2. Sprint Backlog

ID	Sprint Backlog	Status
SB-01	Mencari class yang memiliki duplikasi	DONE
SB-02	Menentukan superclass	DONE
SB-03	Menentukan subclass	DONE
SB-04	Memperbarui class diagram	DONE
SB-05	Implementasi extends	DONE
SB-06	Implementasi super	DONE
SB-07	Melakukan pengujian inheritance	DONE

3. Bagian A - Audit Class

Seluruh class proyek P4 diaudit. Ditemukan 2 class dengan attribute dan behavior yang serupa:

Class	Attribute Umum	Attribute Khusus
Nasabah	nama, alamat	id, rekening
BankSampah	nama, alamat	daftarSampah

Keduanya juga memiliki behavior yang sama:

- `getNama()` dan `getAlamat()`
- `tampilkanData()` yang mencetak nama & alamat
- validasi nama/alamat tidak boleh kosong

Karena Nasabah dan BankSampah sama-sama "entitas" yang memiliki nama dan alamat, maka digeneralisasi:

- Superclass : Entitas (nama, alamat)
- Subclass : Nasabah (id, rekening) dan BankSampah (daftarSampah)

Relasi is-a yang terbentuk:

- Nasabah is-a Entitas
- BankSampah is-a Entitas

4. Bagian B - Class Diagram Terbaru

```
classDiagram
class Entitas {
-String nama
-String alamat
+Entitas(nama, alamat)
+getNama() String
+getAlamat() String
+setNama(nama) void
+setAlamat(alamat) void
+tampilkanIdentitas() void
}
class Nasabah {
```

```

-String id
-Rekening rekening
+Nasabah(id, nama, alamat, saldoAwal)
+getId() String
+getRekening() Rekening
+getSaldo() double
+setSaldo(saldo) void
+tambahSaldo(nominal) void
+tampilkanData() void
}
class BankSampah {
-List~Sampah~ daftarSampah
+BankSampah(nama, alamat)
+getDaftarSampah() List
+getJumlahSampah() int
+tambahSampah(sampah) void
+cariSampah(kode) Sampah
+tampilkanData() void
}
class Sampah {
-String kode
-String jenis
-String nama
-double hargaPerKg
}
class Rekening {
-String nomorRekening
-double saldo
}
class Setoran {
-String idSetoran
-Nasabah nasabah
-Sampah sampah
-double beratKg
}

Entitas <|-- Nasabah : generalization
Entitas <|-- BankSampah : generalization
Nasabah *-- Rekening : composition
BankSampah o-- Sampah : aggregation
Setoran --> Nasabah : association
Setoran --> Sampah : association

```

Relasi yang dipertahankan dari P4:

- Association: Setoran memakai (uses -a) Nasabah dan Sampah.
- Aggregation: BankSampah memiliki (has -a) Sampah.
- Composition: Nasabah memiliki bagian (part -of) Rekening.

Struktur superclass/subclass (pakai diagram teks sebagai pelengkap):

```

+-----+
| Entitas | <-- Superclass (nama, alamat)
+-----+
| - nama |
| - alamat |
+-----+
|
+-----+-----+
| generalization |
v v
+-----+-----+ +-----+-----+
| Nasabah | | BankSampah | <-- Subclass
+-----+-----+ +-----+-----+
| - id | | - daftarSampah |
| - rekening | +-----+
+-----+ o-- Sampah (agg)
| *-- Rekening (comp)
Setoran --> Nasabah & Sampah (association)

```

5. Bagian C - Implementasi Superclass

Dibuat superclass Entitas yang berisi attribute dan behavior yang benar-benar umum (nama, alamat beserta validasi dan getter):

```

public class Entitas {
private String nama;

```

```

private String alamat;

public Entitas(String nama, String alamat) {
    setName(nama);
    setAlamat(alamat);
}

public String getName() {
    return nama;
}

public String getAlamat() {
    return alamat;
}

public void setName(String nama) {
    if (nama != null && !nama.isBlank()) {
        this.nama = nama;
    } else {
        System.out.println("Nama tidak valid");
    }
}

public void setAlamat(String alamat) {
    if (alamat != null && !alamat.isBlank()) {
        this.alamat = alamat;
    } else {
        System.out.println("Alamat tidak valid");
    }
}

public void tampilkanIdentitas() {
    System.out.println("Nama : " + nama);
    System.out.println("Alamat : " + alamat);
}
}

```

6. Bagian D - Implementasi Subclass

Dibuat 2 subclass yang mewarisi Entitas menggunakan extends.

Nasabah (menyimpan attribute khusus: id dan rekening):

```

public class Nasabah extends Entitas {
    private String id;
    private Rekening rekening;

    public Nasabah(String id, String nama, String alamat, double saldoAwal) {
        super(nama, alamat); // memanggil constructor Entitas
        this.id = id;
        this.rekening = new Rekening("R-" + id, saldoAwal);
    }

    public String getId() {
        return id;
    }

    public Rekening getRekening() {
        return rekening;
    }

    public double getSaldo() {
        return rekening.getSaldo();
    }

    public void setSaldo(double saldo) {
        rekening.setSaldo(saldo);
    }

    public void tambahSaldo(double nominal) {
        rekening.tambahSaldo(nominal);
    }

    public void tampilkanData() {
        System.out.println("ID : " + id);
        System.out.println("Nama : " + getName()); // member superclass
        System.out.println("Alamat : " + getAlamat()); // member superclass
        System.out.println("No Rekening : " + rekening.getNomorRekening());
    }
}

```

```

System.out.println("Saldo : Rp" + rekening.getSaldo());
}
}

```

BankSampah (menyimpan attribute khusus: daftar sampah):

```

import java.util.ArrayList;
import java.util.List;

public class BankSampah extends Entitas {
    private List<Sampah> daftarSampah;

    public BankSampah(String nama, String alamat) {
        super(nama, alamat);
        this.daftarSampah = new ArrayList<>();
    }

    public List<Sampah> getDaftarSampah() {
        return daftarSampah;
    }

    public int getJumlahSampah() {
        return daftarSampah.size();
    }

    public void tambahSampah(Sampah sampah) {
        if (sampah != null) {
            daftarSampah.add(sampah);
        } else {
            System.out.println("Object Sampah tidak valid (null)");
        }
    }

    public void tampilkanData() {
        System.out.println("Nama Bank Sampah : " + getName());
        System.out.println("Alamat : " + getAddress());
        System.out.println("Jumlah Sampah Ditampung : " + daftarSampah.size());
        for (Sampah s : daftarSampah) {
            System.out.println(" - " + s.getKode() + " | " + s.getName()
                + " | " + s.getJenis() + " | Rp" + s.getHargaPerKg());
        }
    }

    public Sampah cariSampah(String kode) {
        for (Sampah s : daftarSampah) {
            if (s.getKode().equals(kode)) {
                return s;
            }
        }
        return null;
    }
}

```

7. Bagian E - Implementasi super dan Penjelasan

`super(nama, alamat)` digunakan di baris pertama constructor setiap subclass untuk memanggil constructor superclass `Entitas`.

- Mengapa `super()` digunakan?

Karena attribute `nama` dan `alamat` bersifat `private` di superclass, sehingga tidak dapat diinisialisasi langsung oleh subclass. `super(nama, alamat)` memastikan bagian superclass diinisialisasi dengan benar sebelum subclass menginisialisasi attribute khususnya.

- Constructor mana yang dipanggil?

`Entitas(String nama, String alamat).`

- Urutan eksekusi constructor (constructor chaining):

```

new Nasabah(...)
|-> Constructor Nasabah dijalankan
|-> super(nama, alamat) -> Constructor Entitas dijalankan TERLEBIH DAHULU
|-> id dan rekening diinisialisasi

```

Aturan Java: panggilan `super(...)` harus menjadi pernyataan pertama di constructor subclass. Jika tidak dituliskan, Java akan memanggil constructor `super()` tanpa argumen secara implisit.

Dengan super, duplikasi penulisan nama, alamat, getName(), getAlamat(), dan validasinya di kedua class dihilangkan (di pindahkan ke superclass).

8. Bagian F - Pengujian

Object dibuat dari minimal 2 subclass (Nasabah dan BankSampah), lalu member superclass diakses dari object subclass:

```
Nasabah nasabah1 = new Nasabah("NS-001", "Budi", "Jl. Raya Sumenep 10", 0);
BankSampah bank = new BankSampah("Bank Sampah PENS", "PENS Kampus Sumenep");
```

Hasil eksekusi (Project P5):

```
=== SISTEM MANAJEMEN BANK SAMPAH ===
P5 - Refactoring dengan Inheritance, Generalization, Superclass, dan Subclass

===== P5 - PEMBUATAN OBJECT SUBCLASS =====

[P5.1] Object dibuat dari subclass Nasabah (is-a Entitas).
ID : NS-001
Nama : Budi
Alamat : Jl. Raya Sumenep 10
No Rekening : R-NS-001
Saldo : Rp0.0

ID : NS-002
Nama : Ani
Alamat : Jl. Melati 5
No Rekening : R-NS-002
Saldo : Rp0.0

[P5.2] Object dibuat dari subclass BankSampah (is-a Entitas).
Nama Bank Sampah : Bank Sampah PENS
Alamat : PENS Kampus Sumenep
Jumlah Sampah Ditampung : 0

===== P5 - AKSES MEMBER SUPERCLASS DARI SUBCLASS =====

[P5.3] Subclass memanggil method superclass (getName/getAlamat):
nasabah1.getName() : Budi
nasabah1.getAlamat() : Jl. Raya Sumenep 10
bank.getName() : Bank Sampah PENS
bank.getAlamat() : PENS Kampus Sumenep

[P5.4] Validasi superclass diakses dari subclass:
Nama tidak valid
Nama nasabah1 tetap = Budi
Alamat tidak valid
Alamat bank tetap = PENS Kampus Sumenep

===== P5 - POLYMORPHISM (REFERENSI SUPERCLASS) =====

[P5.5] Referensi bertipe superclass menunjuk ke object subclass:
entitas1 (Nasabah) : Citra - Jl. Angrek 7
entitas2 (BankSampah) : BS Mitra - Jl. Cempaka 2

===== REGRESI P4 : FUNGSIONALITAS TETAP BERJALAN =====

Nama Bank Sampah : Bank Sampah PENS
Alamat : PENS Kampus Sumenep
Jumlah Sampah Ditampung : 2
- SP-001 | Botol Plastik Pet | Anorganik | Rp5000.0
- SP-002 | Koran Bekas | Kertas | Rp2500.0

Setoran ST-001 diproses. Saldo Budi bertambah Rp15000.0
Setoran ST-002 diproses. Saldo Ani bertambah Rp12500.0

[Regresi] cariSampah("SP-002") : Koran Bekas
[Regresi] Setoran memanggil nasabah.getName() : Budi
[Regresi] Composition - Rekening masih dibuat di dalam Nasabah:
No Rekening : R-NS-001
Saldo : Rp15000.0

===== HASIL AKHIR =====

ID : NS-001
Nama : Budi
Alamat : Jl. Raya Sumenep 10
```

No Rekening : R-NS-001
Saldo : Rp15000.0

ID : NS-002
Nama : Ani
Alamat : Jl. Melati 5
No Rekening : R-NS-002
Saldo : Rp12500.0

Regression test P4 (association, aggregation, composition, validasi data tidak valid) tetap bekerja setelah refactoring.

9. Sprint Review & Retrospective

Sprint Review

- Class diagram diperbarui: ya, menampilkan superclass `Entitas`, subclass (`Nasabah`, `BankSampah`), generalization (panah segitiga kosong), attribute, method, serta relasi P4 (association, aggregation, composition).
- Object yang berduplikasi berhasil digeneralisasi: nama, alamat, getter, dan validasinya dipindahkan ke superclass `Entitas`.
- `extends` berhasil diimplementasikan pada 2 subclass; `super(nama, alamat)` berhasil memanggil constructor superclass.
- Tidak ada fitur P4 yang rusak: seluruh relasi dan validasi tetap berjalan (regression test sukses).
- Kendala utama: menentukan nama superclass yang tepat agar relasi is-a tetap masuk akal (dipilih `Entitas` karena subclass bukan keduanya "person" melainkan entitas bernama & berdomisili).

Sprint Retrospective

- What Went Well: Audit class dengan tabel attribute umum vs khusus memudahkan menemukan duplikasi; refactoring bertahap (superclass dulu, subclass setelahnya) membuat kompilasi selalu aman.
- What Went Wrong: Awalnya sempat memikirkan inheritance antara `Setoran` dan `Sampah`, namun hubungan tersebut bertentangan dengan kaidah is-a, sehingga ditinggalkan (hubungan tsb lebih tepat has-a/uses-a).
- Improvement: Ke depan dapat menambahkan inheritance berjenjang (misal subclass dari `Nasabah`), memakai method abstract di superclass, serta memanfaatkan `super.method()` selain `super()` pada constructor.